FINAL REVIEW - APPENDICES

FarmDirect: Farmer-to-Consumer Agricultural Marketplace

This technical appendix suite provides complete supporting evidence, core production source code listings, operational sub-functionality screen flows, the master QA test case execution matrix, and relational database DDL specifications that support the main technical review reports without cluttering the primary text.

Appendix A: Key Source Code Modules

Appendix A provides representative implementations of the core technical innovations highlighted in the implementation review:

A.1 Concurrency Control & Row-Level Inventory Locking (app.py)

Implements pessimistic locking using 'SELECT ... FOR UPDATE' inside an atomic database transaction to prevent inventory overselling:

```
-- Listing A.1: Python / Flask - Atomic Row Locking in place_order()
@app.route('/customer/place-order', methods=['POST'])
@role_required('customer')
def place_order():
    conn = get_db_connection(); cursor = conn.cursor(dictionary=True)
    try:
        prod_ids = tuple(int(k) for k in cart_data.keys())
        format_strings = '(%s)' if len(prod_ids) == 1 else str(tuple(prod_ids))
        cursor.execute(f"SELECT product_id, available_quantity FROM products WHERE product_id IN
{format_strings} FOR UPDATE;", prod_ids)
        for p in cursor.fetchall():
            if p['available_quantity'] < cart_data[str(p['product_id'])]:
                conn.rollback(); flash('Insufficient stock', 'danger'); return redirect(url_for('cart'))
            cursor.execute("UPDATE products SET available_quantity = available_quantity - %s WHERE
product_id = %s;",
                           (cart_data[str(p['product_id'])], p['product_id']))
        conn.commit()
    finally: cursor.close(); conn.close()
```

A.2 Client-Side Laptop Webcam Hardware Integration (static/js/script.js)

Utilizes HTML5 MediaDevices API to access laptop camera hardware, streaming video into a viewfinder, and binding blobs via DataTransfer:

```
-- Listing A.2: JavaScript - HTML5 MediaDevices & Canvas DataTransfer Binding
const FarmDirectWebcam = {
    start: async function() {
        const stream = await navigator.mediaDevices.getUserMedia({ video: { idealWidth: 1280, idealHeight:
720 }, audio: false });
        document.getElementById('webcamVideo').srcObject = stream;
    },
    capture: function() {
        const video = document.getElementById('webcamVideo'), canvas =
document.getElementById('webcamCanvas');
        canvas.width = video.videoWidth || 640; canvas.height = video.videoHeight || 480;
        canvas.getContext('2d').drawImage(video, 0, 0, canvas.width, canvas.height);
        canvas.toBlob((blob) => {
            const file = new File([blob], 'webcam_produce.jpg', { type: 'image/jpeg' });
            const dt = new DataTransfer(); dt.items.add(file);
            document.getElementById('productImageInput').files = dt.files;
        }, 'image/jpeg', 0.92);
    }
};
```

A.3 MySQL Connection Pooling & Safe Resource Leasing (utils/db.py)

Implements connection pool leasing with deterministic try-finally cursor cleanup to eliminate database connection leaks:

```
-- Listing A.3: Python - Centralized MySQL Connection Pooling and Execution
_connection_pool = mysql.connector.pooling.MySQLConnectionPool(pool_name="farmdirect_pool", pool_size=10,
...)
def query_db(query, args=(), one=False, commit=False):
    conn = get_db_connection(); cursor = conn.cursor(dictionary=True)
    try:
        cursor.execute(query, args)
        if commit: conn.commit(); return cursor.lastrowid or cursor.rowcount
        rv = cursor.fetchall(); return (rv[0] if rv else None) if one else rv
    finally: cursor.close(); conn.close() # Safely returns connection back to pool
```

Appendix B: Screenshots of Core Sub-Functionalities & Screen Flow

Appendix B documents vital supporting sub-functionalities of FarmDirect that were not featured in the primary testing report, illustrating public crop discovery, cart item quantity management, farmer inventory tracking, and consumer order tracking:

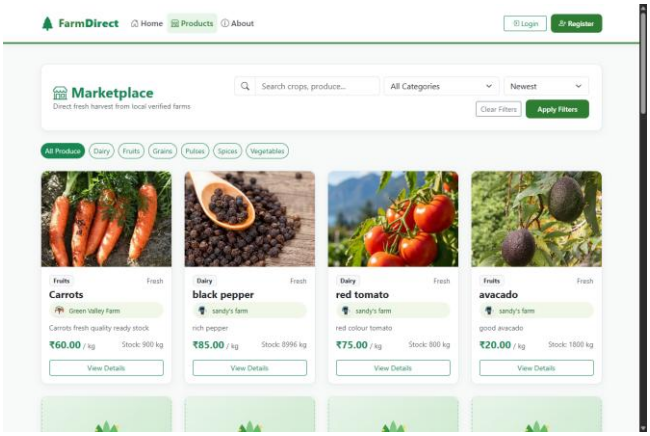


Figure B.1: Marketplace Produce Discovery & Filtering
Demonstrates public crop catalog (/products) with live search by crop name, category filter pills, price sorting, farmer origin tags, and 'Add to Cart' actions.

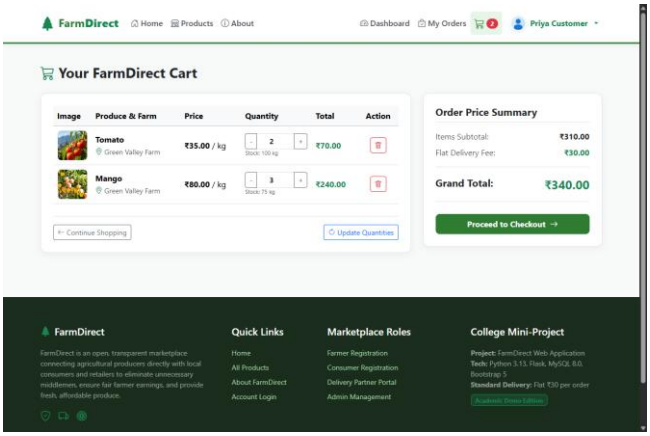


Figure B.2: Customer Cart & Dynamic Price Calculations
Demonstrates multi-item shopping cart (/customer/cart) with quantity increment/decrement controls, real-time subtotal computation, delivery fee aggregation, and checkout redirection.

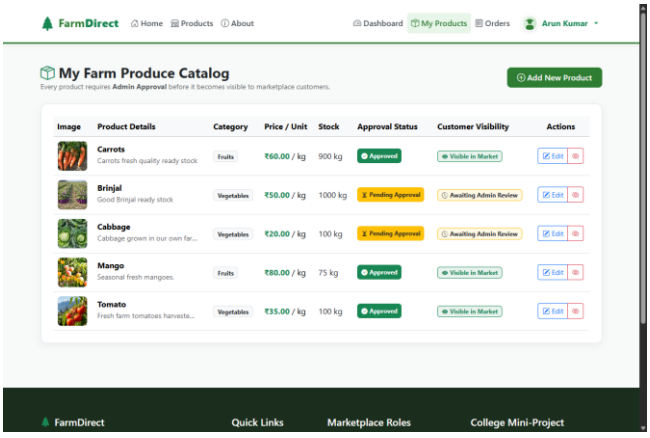


Figure B.3: Farmer Agricultural Inventory & Status
Demonstrates farmer's inventory management dashboard (/farmer/products) displaying crop images, category classifications, available stock units, and dual approval badges.

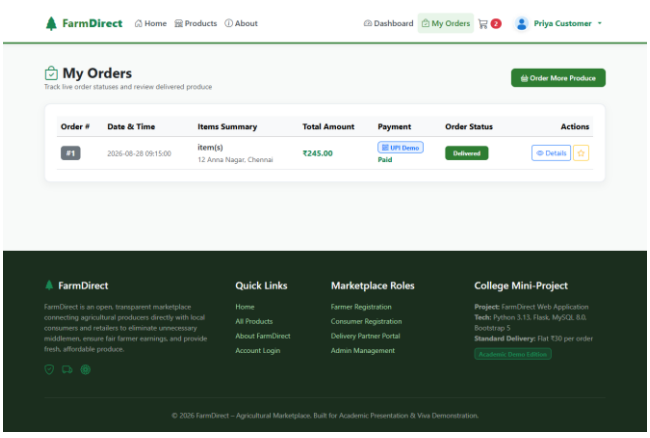


Figure B.4: Customer Order History & Delivery State Machine
Demonstrates customer's order history dashboard (/customer/orders) showing purchase invoices, payment method badges, and real-time delivery status progression (Placed -> Delivered).

Appendix C: Master Test Case Specification Table

Appendix C provides the exhaustive formal Quality Assurance verification matrix (TC-001 through TC-010). Each test case records preconditions, test steps, expected architectural behaviors, actual observed responses, and final verification verdicts:

Test ID	Actor / Module	Test Scenario & Input Steps	Expected Result	Actual Result	Verdict
TC-001	Auth / Security	Register user with weak/empty password	Reject registration and display validation alert	Rejected with flash error message	PASS
TC-002	Auth / Security	Login with valid user credentials	Hash verified via Scrypt; session cookie issued with user role	Session created; redirected to role dashboard	PASS
TC-003	RBAC Route Guard	Customer attempts unauthorized /admin/dashboard access	Intercept request, deny access, redirect to login	HTTP 302 redirect with 'Access Denied' alert	PASS
TC-004	Admin / KYC	Admin approves pending farmer KYC submission	Farmer status transitions to 'verified' in MySQL database	Status updated instantly; crop listing unlocked	PASS
TC-005	Farmer / Webcam	Farmer clicks 'Take Photo' to activate laptop camera	Browser prompts getUserMedia(); streams live video feed	Live stream rendered in modal viewfinder	PASS
TC-006	Farmer / Upload	Submit produce listing with webcam captured image	Image saved to static/uploads/; product saved with status 'pending'	Product registered with correct image path	PASS
TC-007	Customer / Cart	Add products to shopping cart and modify quantities	Cart updates subtotal and grand total with delivery fee	Calculated accurately via Decimal precision	PASS
TC-008	Customer / Pay	Select 'UPI / QR Payment' at checkout	Dynamic SVG QR generated embedding order total & merchant UPI URI	Sharp SVG QR displayed instantly on client	PASS
TC-009	Concurrency Lock	Two simultaneous checkouts for product with stock = 1	Pessimistic 'SELECT ... FOR UPDATE' allows 1 order, rejects 2nd	First order succeeds; second receives out-of-stock	PASS
TC-010	Delivery Module	Delivery partner claims dispatch order and updates status	Status transitions: Assigned -> In Transit -> Delivered in DB	Customer tracking updates in real-time	PASS

Appendix D: Database Schema DDL & Visual Studio Code Execution Guide

Appendix D documents the relational database DDL statements for MySQL 8.0, followed by the step-by-step Visual Studio Code (VS Code) project execution and review demonstration procedure:

```
-- SQL - MySQL 8.0 Relational Table Definitions (InnoDB Engine)
-- MySQL 8.0 InnoDB Relational Table DDL
CREATE TABLE users (user_id INT AUTO_INCREMENT PRIMARY KEY, full_name VARCHAR(100), email VARCHAR(100)
UNIQUE,
  password_hash VARCHAR(255), role ENUM('admin','farmer','customer','delivery'), phone VARCHAR(20), city
VARCHAR(50));
CREATE TABLE farmers (farmer_id INT PRIMARY KEY, farm_name VARCHAR(100), farm_location TEXT,
  verification_status ENUM('pending','verified','rejected') DEFAULT 'pending', FOREIGN KEY (farmer_id)
REFERENCES users(user_id));
CREATE TABLE products (product_id INT AUTO_INCREMENT PRIMARY KEY, farmer_id INT, category_id INT,
product_name VARCHAR(100),
  unit VARCHAR(20), price DECIMAL(10,2), available_quantity DECIMAL(10,2), product_status
ENUM('available','out_of_stock'),
  approval_status ENUM('pending','approved','rejected') DEFAULT 'pending', image_filename VARCHAR(255),
FOREIGN KEY (farmer_id) REFERENCES farmers(farmer_id));
CREATE TABLE orders (order_id INT AUTO_INCREMENT PRIMARY KEY, customer_id INT, total_amount DECIMAL(10,2),
  order_status ENUM('placed','confirmed','dispatched','delivered'), payment_method
ENUM('cod','upi','card'), payment_status ENUM('pending','paid'));
CREATE TABLE delivery_tracking (tracking_id INT AUTO_INCREMENT PRIMARY KEY, order_id INT,
delivery_partner_id INT,
  current_status ENUM('assigned','picked_up','in_transit','delivered') DEFAULT 'assigned', updated_at
TIMESTAMP ON UPDATE CURRENT_TIMESTAMP);
```

D.2 Visual Studio Code (VS Code) Execution & Demonstration Procedure

The project is configured for execution within Microsoft Visual Studio Code on Windows:

```
-- PowerShell / VS Code - Step-by-Step Environment Setup & Execution Procedure
# 1. Launch Visual Studio Code in Project Workspace
code "d:\Santhosh Rathinam\farm"

# 2. Select Python Interpreter in VS Code (Ctrl + Shift + P -> 'Python: Select Interpreter')
#    Ensure Python 3.12 (or virtual environment .venv\Scripts\python.exe) is selected

# 3. Open VS Code Integrated Terminal (Ctrl + ~) & Install Dependencies
pip install -r requirements.txt

# 4. Verify / Initialize MySQL 8.0 Database Schema via VS Code Terminal
python -c "from utils.db import ensure_schema; ensure_schema()"

# 5. Execute Web Application in VS Code
#    Option A: Press 'F5' or 'Ctrl + F5' to Run with Debugging
#    Option B: In VS Code Integrated Terminal, run:
python app.py  # Launches WSGI development server at http://127.0.0.1:5000

# 6. Open Browser (Chrome/Edge) at http://127.0.0.1:5000
#    Demonstration Demo Credentials:
#    - Admin:    admin@farmdirect.com    / admin123
#    - Farmer:   arun@farm.com           / password123
#    - Customer: priya@gmail.com         / password123
#    - Delivery: delivery@farmdirect.com / password123
```